

Federated Learning for Aircraft Engine Health Monitoring

Technical Methodology,
Implementation, and
Results

Mustakim Rahman

June 30, 2026

Contents

1	Summary	3
2	Requirement Traceability	3
3	Problem Formulation	4
4	Dataset and Preprocessing	4
4.1	Dataset	4
4.2	Label Construction	5
4.3	Leakage Control	5
5	Model and Training Design	5
5.1	Dual-Head CNN-GRU	5
5.2	Objective Function	5
5.3	Federated Optimization	6
6	Evaluation Metrics	6
7	Run the Experiments	6
7.1	Environment	6
7.2	One-Command Experiment Run	6
8	Task 1 Results: Baseline Federated Learning	7
8.1	Compared Training Regimes	7
8.2	Interpretation	7
9	Task 2 Results: Research Questions	7
9.1	RQ1: Heterogeneous Sensor Availability	7
9.1.1	Problem and Solve Procedure	7
9.1.2	Interpretation	8
9.2	RQ2: Class Imbalance Across Clients	8
9.2.1	Problem and Solve Procedure	8
9.2.2	Interpretation	9
9.3	RQ3: Root Cause and Interpretability	9
9.3.1	Problem and Solve Procedure	9
9.3.2	Interpretation	10
9.4	RQ4: Concept Drift	10
9.4.1	Problem and Solve Procedure	10
9.4.2	Interpretation	10
9.5	RQ5: Validation Mismatch Under Non-IID Clients	10
9.5.1	Problem and Solve Procedure	10
9.5.2	Interpretation	10
9.6	RQ6: Membership Inference From Client Updates	11
9.6.1	Threat Model and Solve Procedure	11
9.6.2	Interpretation	11
9.7	RQ7: Model Poisoning	12
9.7.1	Threat Model and Solve Procedure	12
9.7.2	Interpretation	12

10 Codebase Map	13
11 Limitations	13
12 Future Research Directions	13
12.1 Personalized Federated PHM	13
12.2 Dynamic Sensor Missingness	14
12.3 Drift-Aware FL Control	14
12.4 Formal Privacy-Preserving FL	14
12.5 PHM-Specific Robust Aggregation	14
13 Conclusion	14

1 Summary

This document reports a complete implementation of the aircraft-engine federated learning assignment described in [ProjectforPhDAplicants.pdf](#). The project asks for a privacy-preserving prognostics and health management (PHM) pipeline in which multiple airline operators train an engine-health model collaboratively without sharing raw sensor trajectories.

The implemented solution uses NASA C-MAPSS FD001 [1], partitions complete engine units across five simulated airline clients, and trains a dual-head CNN-GRU model for two coupled tasks: Remaining Useful Life (RUL) regression and early fault detection. The baseline uses Federated Averaging (FedAvg) [2]. The project then implements all seven research questions from the brief: heterogeneous sensor availability, class imbalance, interpretability, concept drift, validation mismatch, membership inference, and model poisoning.

The strongest baseline result is the validation-best federated model, with RMSE 14.024, MAE 10.191, AUROC 0.9972, AUPRC 0.9207, precision 0.8821, recall 0.7440, and F1 0.8072. This improves substantially over isolated client training, whose client-macro RMSE is 39.524 and MAE is 35.906. The result supports the core hypothesis: cross-client model sharing improves aircraft engine prognostics while preserving local ownership of raw sensor data.

2 Requirement Traceability

Table 1 maps the brief requirements to the implemented repository artifacts and experimental evidence.

Table 1: Requirement traceability matrix.

Brief item	Technical requirement	Status	Primary evidence
Task 1	Use an aviation-relevant dataset, 3–6 clients, one central aggregation server, and a model for RUL and early fault detection.	Implemented	<code>data/preprocessing.py</code> , <code>models/dual_head_model.py</code> , <code>federated/</code> , <code>baselines/</code> , <code>experiments/logs/summary_results.csv</code>
Task 2	Select one or more RQ1–RQ7 and implement suitable solutions.	All seven implemented	<code>research_questions/</code> , <code>experiments/run_rq*.py</code> , <code>configs/task2*.yaml</code> , <code>experiments/logs/rq*_summary_results.csv</code>
Task 3	Identify research gaps, future directions, and potential solutions.	Implemented	<code>reports/task3_future_directions.md</code> ; summarized in Section 12

3 Problem Formulation

Aircraft PHM estimates degradation state from multivariate engine sensor time series. The assignment frames the practical problem as one of both predictive performance and data governance. A single airline may have too few failure examples to train a robust model, but centralizing raw sensor data can expose commercially sensitive information such as maintenance schedules, fleet condition, and capacity risk.

The implemented learning problem has two outputs:

1. **RUL regression:** estimate the number of cycles remaining before failure or maintenance threshold.
2. **Early fault detection:** classify whether a time window is within the near-failure region.

Federated learning is an appropriate solution because it moves model training to the client data, exchanges model updates rather than raw trajectories, and aggregates updates into a shared global model [2]. This directly matches the cross-airline collaboration constraint in the brief.

Central research question: Can multiple airline clients train a shared engine-health model that improves RUL and fault prediction compared with isolated local training, while keeping raw sensor trajectories local to each client?

4 Dataset and Preprocessing

4.1 Dataset

The implementation uses NASA C-MAPSS FD001, a standard turbofan engine degradation benchmark for PHM research [1]. The repository expects the following local files:

```
data/raw/CMAPSSData/train_FD001.txt
data/raw/CMAPSSData/test_FD001.txt
data/raw/CMAPSSData/RUL_FD001.txt
```

Table 2: Dataset and partitioning configuration.

Parameter	Value
Subset	FD001
Simulated clients	5 airline operators
Client partitioning	Complete engine units, IID round-robin by shuffled unit ID
Validation split	20% of training engine units
Window size / stride	30 cycles / 1 cycle
RUL cap	125 cycles
Fault threshold	raw RUL $\leq$ 30 cycles
Sensor channels	s2, s3, s4, s7, s8, s9, s11, s12, s13, s14, s15, s17, s20, s21

4.2 Label Construction

For engine unit u at cycle t , the preprocessing code computes:

$$\text{rawRUL}_{u,t} = \max(\text{cycle}_u) - t, \quad (1)$$

$$y_{u,t}^{\text{rul}} = \min(\text{rawRUL}_{u,t}, 125), \quad (2)$$

$$y_{u,t}^{\text{fault}} = \begin{cases} 1, & \text{rawRUL}_{u,t} \leq 30, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

RUL capping is used because very early-life exact RUL values are less informative than the later degradation trajectory. The binary fault label creates an operational near-failure target for maintenance triage.

4.3 Leakage Control

Complete engine units, not individual windows, are assigned to validation and client partitions. This matters because random window splitting would leak adjacent windows from the same engine trajectory across train, validation, and client boundaries. Sensor scaling is fitted on the training units only and then applied to validation and test data. These choices are implemented in `data/preprocessing.py`.

5 Model and Training Design

5.1 Dual-Head CNN-GRU

The model in `models/dual\_head\_model.py` combines convolutional feature extraction with a gated recurrent unit (GRU) sequence encoder [3]. The convolutional layers learn local temporal patterns and cross-sensor interactions; the GRU summarizes the 30-cycle degradation window into one hidden state. Two heads share this encoder:

- a scalar RUL regression head;
- a scalar fault-logit classification head.

Table 3: Baseline model overview.

Component	Configuration
Input shape	batch $\times$ 30 cycles $\times$ 14 sensors
Convolution stack	Conv1d 64, Conv1d 128, BatchNorm, ReLU, Dropout
Sequence layer	GRU hidden size 128
Shared post-processing	LayerNorm, Dropout 0.2
RUL head	Linear output, one scalar
Fault head	Linear output, one logit

5.2 Objective Function

Training minimizes a multi-task loss in `federated/client.py`:

$$\mathcal{L} = \text{MSE}(\hat{y}^{\text{rul}}, y^{\text{rul}}) + \lambda_{\text{fault}} \text{BCEWithLogits}(\hat{z}^{\text{fault}}, y^{\text{fault}}). \quad (4)$$

This is the correct objective for the problem because the project requires continuous RUL estimation and binary early-fault classification from the same sensor window. Sharing the encoder lets both tasks use a common degradation representation while preserving task-specific outputs.

5.3 Federated Optimization

The baseline server implements sample-weighted FedAvg [2]:

$$\mathbf{w}^{(t+1)} = \sum_{k=1}^K \frac{n_k}{\sum_{j=1}^K n_j} \mathbf{w}_k^{(t+1)}, \quad (5)$$

where $\mathbf{w}^{(t)}$ is the global model at round t , $\mathbf{w}_k^{(t+1)}$ is client k 's locally trained model, and n_k is the number of local training windows. The implementation uses all five clients in each round. Local training uses Adam, mini-batches, and the loss in Equation 4.

6 Evaluation Metrics

RUL performance is reported with RMSE, MAE, and the asymmetric NASA PHM score. The NASA score penalizes optimistic late predictions more heavily:

$$s_i = \begin{cases} \exp(-d_i/13) - 1, & d_i < 0, \\ \exp(d_i/10) - 1, & d_i \geq 0, \end{cases} \quad d_i = \hat{y}_i - y_i. \quad (6)$$

Fault detection is reported with AUROC, AUPRC, precision, recall, F1, and full confusion counts. AUPRC is especially important because near-failure windows are rare relative to healthy windows.

7 Run the Experiments

7.1 Environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Required Python packages are `numpy`, `pandas`, `scikit-learn`, `torch`, and `PyYAML`.

7.2 One-Command Experiment Run

The complete implemented pipeline is executed by:

```
./run_all.sh
```

This sequentially runs Task 1 and RQ1–RQ7. CSV logs are written to `experiments/logs/`. For more details like running any individual task, with or without specific parameter, see `README.md` of the repository

8 Task 1 Results: Baseline Federated Learning

8.1 Compared Training Regimes

- **Isolated:** each client trains a separate local model.
- **Centralized:** all client windows are pooled in one central model.
- **Federated:** clients train locally and the server aggregates with FedAvg.

Table 4: Task 1 baseline results.

Method	RMSE	MAE	AUROC	AUPRC	Prec.	Recall	F1
Isolated mean	39.524	35.906	0.9785	0.7107	0.6377	0.6380	0.5866
Centralized	16.167	11.693	0.9964	0.8999	0.7881	0.8404	0.8134
Federated best	14.024	10.191	0.9972	0.9207	0.8821	0.7440	0.8072

Isolated mean denotes the macro-average performance across the five independently trained client models (no parameter sharing). Centralized denotes a single model trained using the combined data from all clients. Federated best reports the best-performing federated learning configuration among the evaluated aggregation methods and hyperparameter settings.

8.2 Interpretation

The isolated client models perform substantially worse because each client sees only a subset of the fleet degradation trajectories. Both centralized and federated training benefit from cross-client signal. The federated model obtains the best RMSE, MAE, AUROC, AUPRC, and precision in the current logs while never pooling raw client windows. Centralized training has higher recall at the fixed 0.5 threshold, so operational deployment would require threshold calibration depending on whether the maintenance policy prioritizes false-alarm control or fault capture.

9 Task 2 Results: Research Questions

9.1 RQ1: Heterogeneous Sensor Availability

9.1.1 Problem and Solve Procedure

Standard FL assumes all clients expose the same input dimension. Real airline fleets may log different sensors. The implemented solution compares:

1. a lowest-common-sensor baseline, which keeps only sensors shared by all clients;
2. a variable-sensor adapter model, where each client maps its own sensor subset into a common representation before the shared CNN-GRU.

This directly solves the input-heterogeneity problem because the shared model can operate on a common representation while client-specific adapters handle local sensor layouts.

Table 5: RQ1 simulated sensor availability by client.

Client	Sensors	Sensor list	Train windows
0	14	s2 s3 s4 s7 s8 s9 s11 s12 s13 s14 s15 s17 s20 s21	2650
1	10	s2 s3 s4 s7 s11 s12 s15 s17 s20 s21	2939
2	6	s2 s4 s7 s11 s15 s20	2788
3	10	s3 s4 s8 s9 s11 s13 s14 s15 s17 s21	3086
4	7	s2 s7 s11 s12 s15 s20 s21	2559

Table 6: RQ1 final results.

Method	Sensors	RMSE	MAE	AUPRC	Recall	F1
Lowest common sensors	2 common	19.052	14.585	0.8187	0.7229	0.7262
Variable sensor adapter	client-specific	21.624	16.620	0.6341	0.5361	0.4821

9.1.2 Interpretation

The adapter design is the correct architectural response to heterogeneous inputs, but the current run shows that the lowest-common-sensor baseline performs better. This is a useful research finding: the adapter model adds local parameters and personalization complexity, and its client-macro result suffers when some clients have only six or seven sensors. Future versions should use stronger adapter pretraining, masked sensor-token attention, or shared adapter regularization.

9.2 RQ2: Class Imbalance Across Clients

9.2.1 Problem and Solve Procedure

Fault-positive windows are rare. If standard FedAvg weights only by sample count, rare failure gradients can be diluted. RQ2 therefore implements two remedies: positive-class weighted BCE and fault-aware aggregation. The weighted BCE term is based on the standard imbalance correction idea used in binary classification, while SMOTE-style oversampling is included as an exploratory baseline [4].

Table 7: RQ2 per-client fault distribution.

Client	Train windows	Fault windows	Fault rate	BCE w^+
0	2746	496	0.1806	4.5363
1	2514	496	0.1973	4.0685
2	2639	496	0.1879	4.3206
3	3044	496	0.1629	5.1371
4	3079	496	0.1611	5.2077

Table 8: RQ2 final test results.

Method	RMSE	MAE	AUROC	AUPRC	Prec.	Recall	F1
FedAvg	13.631	9.824	0.9978	0.9392	0.8654	0.8133	0.8385
FedAvg + weighted BCE	13.771	9.532	0.9974	0.9294	0.7769	0.9337	0.8482
Fault-aware aggregation	13.640	9.705	0.9976	0.9339	0.8495	0.8163	0.8326
Fault-aware + weighted BCE	14.054	9.893	0.9974	0.9296	0.7086	0.9669	0.8178
SMOTE FedAvg	13.396	9.332	0.9972	0.9228	0.7851	0.8916	0.8350
KMeans-SMOTE FedAvg	13.335	9.093	0.9972	0.9241	0.7714	0.8946	0.8285

9.2.2 Interpretation

Weighted BCE is the most effective mechanism for increasing fault recall. The highest recall is 0.9669 for fault-aware aggregation plus weighted BCE, compared with 0.8133 for standard FedAvg. The cost is lower precision, which means more false alarms. KMeans-SMOTE obtains the best RMSE and MAE, but synthetic interpolation of sequential degradation windows may not preserve engine physics; therefore it is best treated as a comparison baseline rather than the primary PHM solution.

9.3 RQ3: Root Cause and Interpretability

9.3.1 Problem and Solve Procedure

Maintenance engineers need more than a predicted RUL value. RQ3 computes input-gradient attributions [5] for selected high-risk test windows, then maps influential sensors to a lightweight maintenance ontology:

$$a_s = \frac{1}{T} \sum_{t=1}^T \left| x_{t,s} \frac{\partial o}{\partial x_{t,s}} \right|, \quad (7)$$

where o is either the RUL output or the fault logit. This is done because it connects model behavior to subsystem-level inspection context, making predictions more actionable for engineering review.

Table 9: RQ3 model performance and explanation coverage.

Attribution	Windows	RMSE	MAE	AUPRC	Prec.	F1
Input gradient	128	14.124	10.108	0.9118	0.9306	0.7903

Table 10: RQ3 top fault-head sensor attributions.

Rank	Sensor	Subsystem	Mean abs. grad.	Norm. imp.
1	s12	bypass and flow path	0.0649	0.1455
2	s11	bypass and flow path	0.0574	0.1287
3	s2	thermal core	0.0547	0.1227
4	s13	spool speed control	0.0339	0.0760
5	s7	compressor flow path	0.0331	0.0741

9.3.2 Interpretation

The strongest fault-head explanations emphasize bypass/flow-path and thermal-core sensors. This is useful for triage, but it is not proof of physical causality. The ontology should be treated as decision support requiring validation by maintenance-domain experts.

9.4 RQ4: Concept Drift

9.4.1 Problem and Solve Procedure

The brief asks whether an injected client-specific drift degrades local AUPRC and whether a lightweight trigger can recover within ten FL rounds. The experiment injects drift at round 15 for client 0, monitors local performance, and triggers recovery if AUPRC falls below a configured fraction of its recent baseline. This is the correct test because concept drift is local to a client in cross-silo airline FL, not necessarily global.

Table 11: RQ4 concept-drift summary.

Drift client	Drift round	Detection	Recovered	Final drift metric
0	15	not detected	false	0.9862

9.4.2 Interpretation

The injected drift was not detected in the current run. This is a valid negative result: the configured drift or detection threshold did not create a monitored AUPRC drop large enough to fire the trigger. Future work should use pre/post drift feature-distribution tests, residual calibration drift, and per-client control charts rather than a single AUPRC drop rule.

9.5 RQ5: Validation Mismatch Under Non-IID Clients

9.5.1 Problem and Solve Procedure

Cross-client validation can confuse distribution mismatch with update quality. RQ5 compares naive validation weighting against distribution-aware weighting based on client feature and label summary similarity. This is done because a model trained on one operating regime can score poorly on another client for valid distributional reasons, not because its update is harmful.

Table 12: RQ5 validation-weighting results.

Method	RMSE	MAE	AUROC	AUPRC	Prec.	Recall	F1
Naive validation	14.705	10.917	0.9965	0.8956	0.8031	0.7861	0.7945
Distribution-aware validation	15.661	12.072	0.9958	0.8706	0.8241	0.7620	0.7919

9.5.2 Interpretation

Distribution-aware validation reduced false positives and improved precision (0.8241 vs. 0.8031), but naive validation produced better RMSE, MAE, AUPRC, recall, and F1 in

this run. The result shows that similarity correction is not automatically beneficial; the similarity metric must be carefully calibrated to distinguish meaningful operating-regime difference from poor model quality.

9.6 RQ6: Membership Inference From Client Updates

9.6.1 Threat Model and Solve Procedure

Federated learning hides raw data, but model updates can leak membership information [6, 7]. RQ6 evaluates a passive attacker observing the target client’s pre-update global model and post-local-training model. Member samples are target-client training windows; non-members are C-MAPSS test windows.

Two attacks are implemented:

$$\text{score}_{\text{loss}}(x) = -\mathcal{L}(\mathbf{w}_{\text{local}}, x), \quad (8)$$

$$\text{score}_{\text{update}}(x) = \mathcal{L}(\mathbf{w}_{\text{global}}, x) - \mathcal{L}(\mathbf{w}_{\text{local}}, x). \quad (9)$$

Defenses use whole-update clipping and clipping plus Gaussian noise, inspired by differential privacy mechanisms [8, 9]. No formal privacy budget is claimed.

Table 13: RQ6 loss-threshold membership inference results.

Defense	Noise	Attack AUROC	Attack acc.	Test RMSE	Test AUPRC	Test F1
No defense	0.0	0.5162	0.5255	14.216	0.9031	0.7905
Update clipping	0.0	0.6727	0.6490	60.021	0.8037	0.7264
Clipping + noise	0.1	0.6790	0.6490	63.467	0.8032	0.7239
Clipping + noise	0.3	0.6739	0.6490	60.637	0.8064	0.7349
Clipping + noise	0.5	0.6718	0.6495	63.840	0.8175	0.7438
Clipping + noise	1.0	0.6761	0.6485	63.580	0.8107	0.7298

Table 14: RQ6 update-effect membership inference results.

Defense	Noise	Attack AUROC	Attack acc.	Test RMSE	Test AUPRC	Test F1
No defense	0.0	0.6223	0.5965	14.216	0.9031	0.7905
Update clipping	0.0	0.6908	0.6480	60.021	0.8037	0.7264
Clipping + noise	0.1	0.6910	0.6480	63.467	0.8032	0.7239
Clipping + noise	0.3	0.6810	0.6475	60.637	0.8064	0.7349
Clipping + noise	0.5	0.6803	0.6475	63.840	0.8175	0.7438
Clipping + noise	1.0	0.6917	0.6480	63.580	0.8107	0.7298

9.6.2 Interpretation

The update-effect attack exposes measurable membership signal even without defense (AUROC 0.6223). The implemented clipping/noise settings do not reduce attack AUROC; in this run they increase attack separability and severely degrade RUL utility. This negative result is important: privacy defense must be designed with formal accounting, calibrated clipping, and possibly secure aggregation, not ad hoc whole-model perturbation. Empty cells in the raw RQ6 CSV are “not applicable” fields because loss-threshold and update-effect attacks report different diagnostic quantities.

9.7 RQ7: Model Poisoning

9.7.1 Threat Model and Solve Procedure

RQ7 evaluates a malicious airline client that corrupts near-failure labels before local training, increasing RUL targets and flipping fault labels to healthy. This is a direct PHM poisoning attack because it tries to make risky engines look healthier. Defenses include update clipping, coordinate median, trimmed mean, and a detection filter. Median and trimmed mean are standard robust aggregation ideas for Byzantine-resilient distributed learning [10].

Table 15: RQ7 poisoning and defense results. NF recall = near-failure fault recall.

Method	Poisoned	RMSE	AUPRC	Prec.	Recall	F1	NF RMSE	NF recall
Clean FedAvg	no	14.307	0.9096	0.8941	0.6867	0.7768	6.999	0.6867
Poisoned FedAvg	yes	15.103	0.8577	1.0000	0.0904	0.1657	19.607	0.0904
Poisoned + clipping	yes	67.998	0.7996	0.9080	0.4759	0.6245	13.364	0.4759
Poisoned + median	yes	14.603	0.8882	0.9403	0.3795	0.5408	13.108	0.3795
Poisoned + trimmed mean	yes	16.103	0.8425	0.9194	0.3434	0.5000	16.524	0.3434
Poisoned + detection filter	yes	14.765	0.9043	0.9087	0.6596	0.7644	7.095	0.6596

9.7.2 Interpretation

Poisoned FedAvg sharply reduces recall from 0.6867 to 0.0904, even though precision becomes 1.0000 because the model predicts very few faults. The detection filter is the strongest defense in this run: it nearly restores clean FedAvg behavior, with RMSE 14.765, AUPRC 0.9043, F1 0.7644, and near-failure recall 0.6596. Update clipping recovers some recall but heavily damages RUL utility. This demonstrates why security evaluation in PHM must report near-failure metrics, not only aggregate RMSE or AUROC.

10 Codebase Map

Table 16: Primary implementation modules.

Path	Role
<code>data/preprocessing.py</code>	C-MAPSS loading, RUL/fault labels, scaling, engine-unit splits, sliding windows, client partitioning.
<code>models/dual_head_model.py</code>	Shared CNN-GRU encoder with RUL and fault heads.
<code>models/heterogeneous_sensor_model.py</code>	Client-specific adapters for variable sensor availability.
<code>federated/client.py</code>	PyTorch dataset, local training, multitask loss, prediction, evaluation.
<code>federated/server.py</code>	FedAvg state-dictionary aggregation.
<code>federated/fl_runner.py</code>	Baseline federated training loop.
<code>baselines/</code>	Isolated and centralized comparisons.
<code>research_questions/</code>	RQ1–RQ7 experimental methods.
<code>experiments/run_rq*.py</code>	Executable entry points for each research question.
<code>evaluation/metrics.py</code>	RMSE, MAE, NASA score, AUROC, AUPRC, precision, recall, F1, confusion counts.

11 Limitations

1. **Dataset scope.** All experiments use C-MAPSS FD001. Validation on FD002–FD004, N-CMAPSS, or airline-specific data would improve external validity.
2. **Fixed threshold.** Fault classification uses a 0.5 threshold. Several results show that threshold calibration is essential for safety-cost trade-offs.
3. **Heuristic privacy defense.** RQ6 clipping and noise are DP-inspired only; no formal ϵ, δ guarantee is claimed.
4. **Interpretability scope.** RQ3 attribution explains model sensitivity, not physical causality.
5. **Synthetic oversampling.** SMOTE and KMeans-SMOTE may generate degradation windows that are statistically useful but physically questionable.
6. **Prototype status.** These experiments are research evidence and are not aviation certification evidence.

12 Future Research Directions

12.1 Personalized Federated PHM

A shared encoder with client-specific prediction heads could handle airline differences in routes, climates, maintenance practices, and fleet age better than one fully shared model.

12.2 Dynamic Sensor Missingness

RQ1 should be extended from fixed per-client sensor subsets to inference-time missingness. A sensor-token model with missingness masks can use any available sensor without retraining the full architecture.

12.3 Drift-Aware FL Control

RQ4 should combine AUPRC monitoring with feature-distribution statistics, calibration error, residual drift, and multi-round control limits. Recovery should include client personalization and temporary aggregation down-weighting.

12.4 Formal Privacy-Preserving FL

RQ6 should be extended to differentially private FedAvg with per-layer clipping, secure aggregation, and privacy accounting. Membership inference should remain as an audit metric after defense.

12.5 PHM-Specific Robust Aggregation

RQ7 shows that poisoning effects are clearest near failure. Future robust aggregation should optimize not just global validation metrics but also near-failure recall, healthy-bias, and threshold stability.

13 Conclusion

The project implements the complete requested baseline and all seven research questions from the PhD applicant brief. The strongest practical result is that the federated model substantially outperforms isolated client training while keeping raw sensor windows local. The research-question experiments also expose important engineering realities: heterogeneous sensor adaptation is not automatically better than common-sensor baselines, imbalance correction improves recall at a precision cost, simple drift triggers can miss injected drift, distribution-aware validation needs careful calibration, naive privacy defenses can worsen both utility and leakage, and poisoning defenses must be evaluated on near-failure behavior.

Overall, the repository provides a reproducible research prototype for privacy-preserving aircraft engine PHM and a technically grounded basis for further doctoral-level investigation.

References

- [1] A. Saxena and K. Goebel. *Turbofan Engine Degradation Simulation Data Set*. NASA Prognostics Data Repository, NASA Ames Research Center, 2008.
- [2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of AISTATS*, 2017.
- [3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of EMNLP*, 2014.

- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [5] K. Simonyan, A. Vedaldi, and A. Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *ICLR Workshop*, 2014.
- [6] R. Shokri, M. Stronati, C. Song, and V. Shmatikov. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy*, 2017.
- [7] L. Bai, H. Hu, Q. Ye, H. Li, L. Wang, and J. Xu. Membership inference attacks and defenses in federated learning: A survey. *ACM Computing Surveys*, 57(4), Article 89, 2024.
- [8] C. Dwork, F. McSherry, K. Nissim, and A. Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography Conference*, 2006.
- [9] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. Deep learning with differential privacy. In *ACM SIGSAC Conference on Computer and Communications Security*, 2016.
- [10] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, 2018.